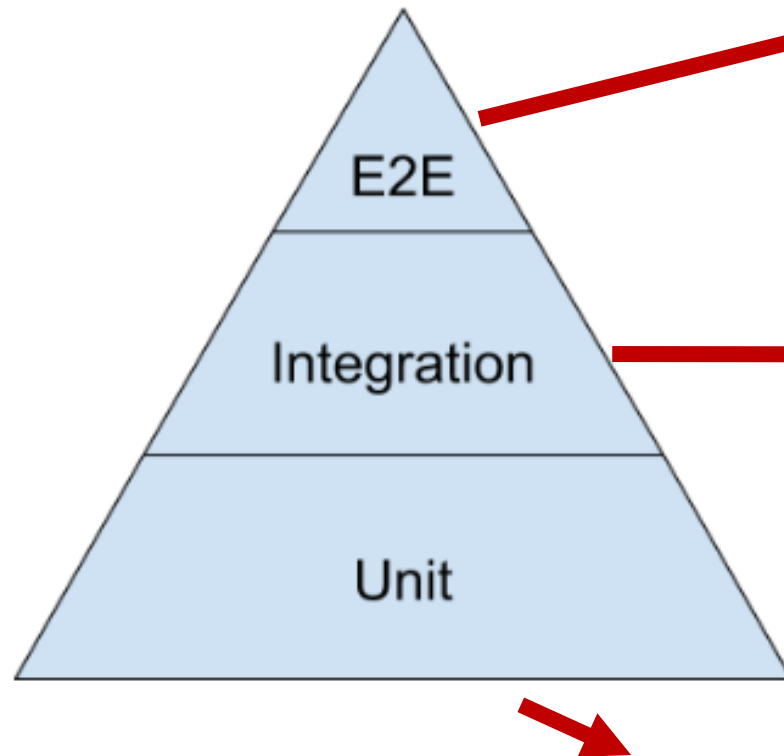


Software Testing
Course's Code: CSE 453
Automated Tool Support for Testing
(Automated Mock Framework)[Chapter 5]

What is a test tool

- ❑ **A test tool is a software product that supports one or more test activities**, such as planning and control, specification, building initial files and data, test execution and test analysis.
- ❑ **Types of Test Tool**
 - Test Management Tools
 - Requirements Management Tool
 - Configuration Management Tool
 - **Test Execution Tool -> Testing with code [In this course]**
 - Test Design Tool
 - Record (or capture playback) tools -> Codeless Testing

Automated Tool Support towards Different levels of testing pyramid



- Junit, TestNG, Mockito for Java language
- Jest for java script
- Nunit for .net languages
- pytest for python

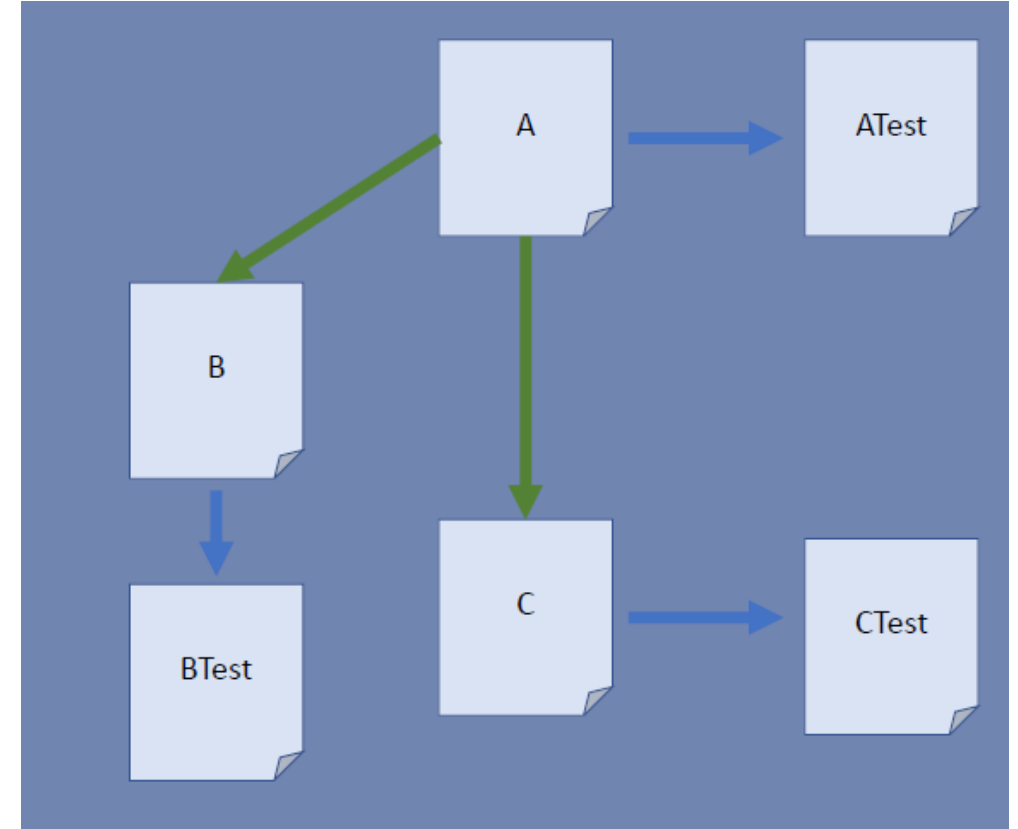
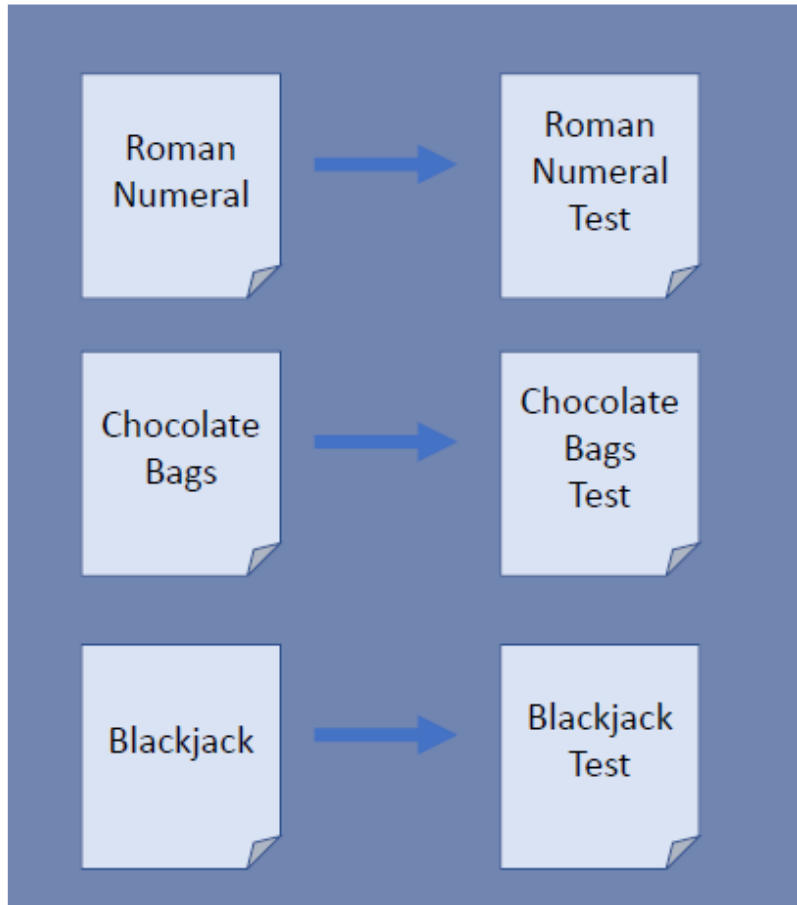
- Selenium, Cypress, Playwright, Appium
- Postman for API testing tool manually,
- Rest Assured for automated API testing tool, [This also follow BDD]
- Pact for Consumer Driven Contract testing tool,
- Cucumber follows BDD [Behavior Driven Development]
- Mockito to test interfaces between different components of a software
- WireMocks for API testing

- **This is not the Complete List**
- **More tools exist**
- **Go for a search**

Consideration during Automated Testing

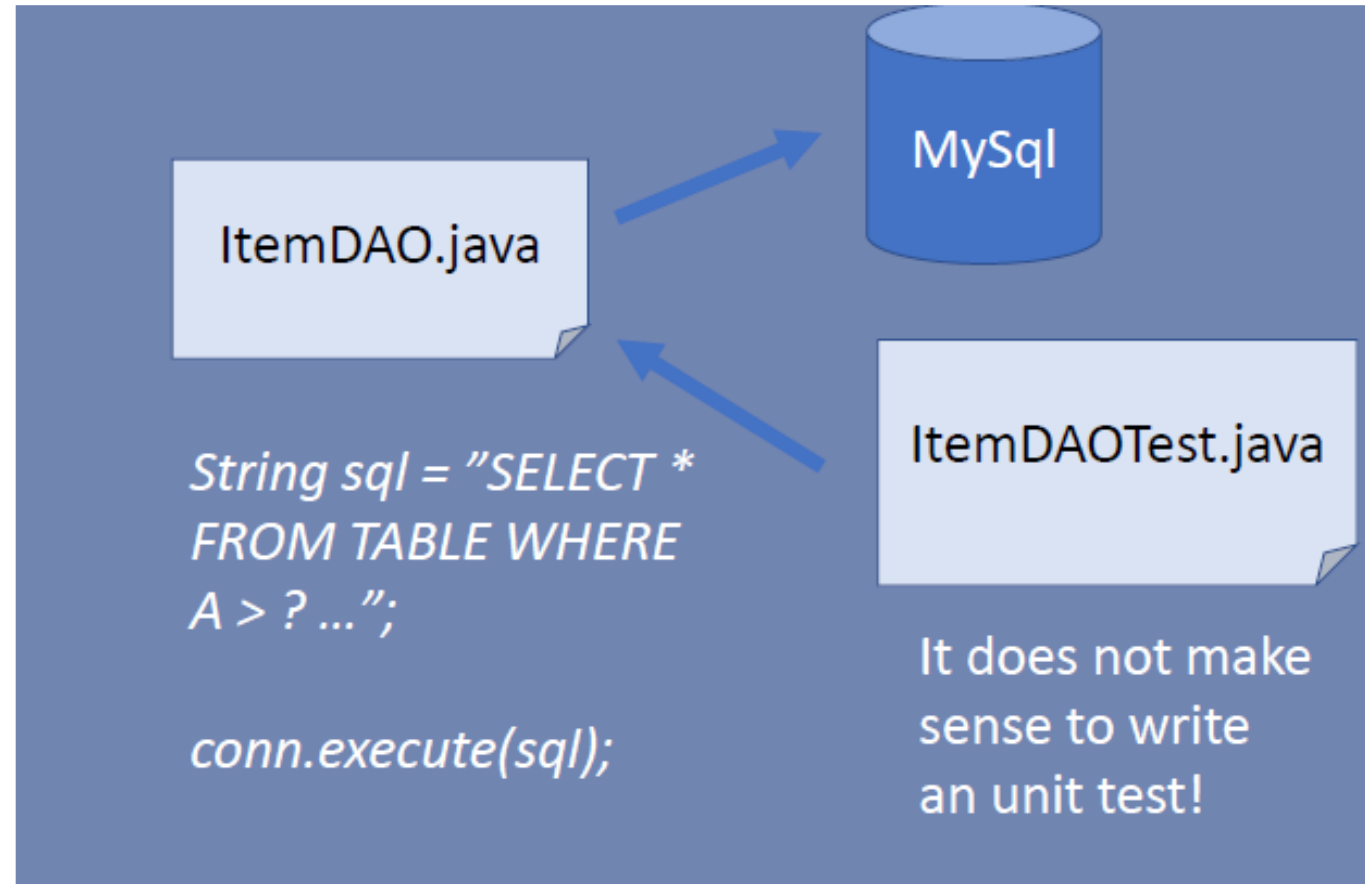
- ❑ **Developing software to test the software is called test automation/automated testing**
- ❑ Clear and reasonable expectation should be established in order to know what can and what can not be accomplished with automated testing
- ❑ There should be a clear understanding of the requirements that should be met in order to achieve successful automated testing
- ❑ Select a tool that allows the implementation of automated testing in a way that conforms to the specified long-term testing strategy

Unit Testing



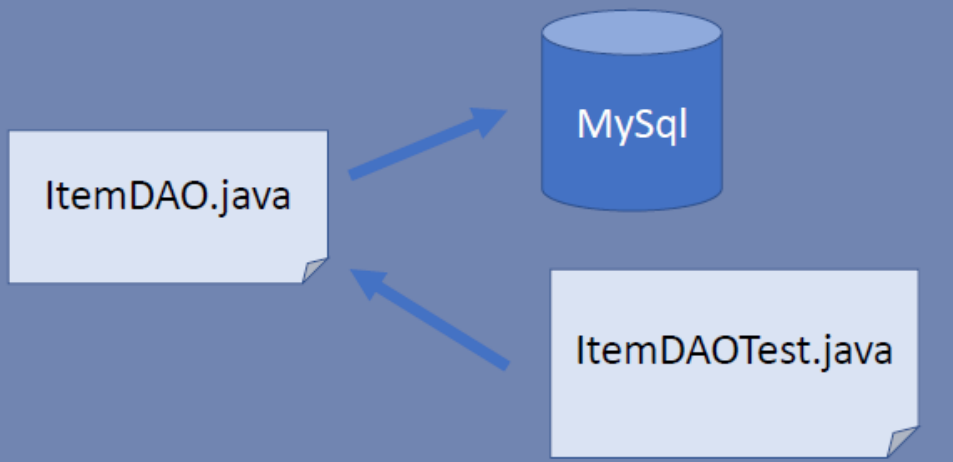
Integration Testing

- *ISQTB: “Tests interfaces between components, interactions to different part of a system such as OS, file system, hardware or interfaces between systems.”*



Integration Testing

- We need a DB!
- Make sure the DB has the right schema
- Set up the database state (INSERTs, ...)
- Make sure one test does not interfere in the other
 - Clean up everything after the test



- There's always more work to do when it's an integration test.
- If you're doing integration tests for a web service, for example, you will face similar problems.
- You will need to make sure that your web service is available, that your web service is in a state that you can execute this.
- Integration tests are very useful but also harder to write.
- It is the task of a software tester to decide when to write integration tests, and when not to write integration tests

Example of Integration Testing

I wanna filter all the invoices where their value are smaller than 100.0.
Invoices come from the database.



```
public List<Invoice> filter() {  
    InvoiceDao dao = new InvoiceDao();  
    List<Invoice> all= dao.all();  
    List<> filtered = new ArrayList<>();  
  
    for(Invoice inv : all) {  
        if(inv.getValue() < 100.0)  
            filtered.add(inv);  
    }  
  
    return filtered;  
}
```



Example of Integration Testing

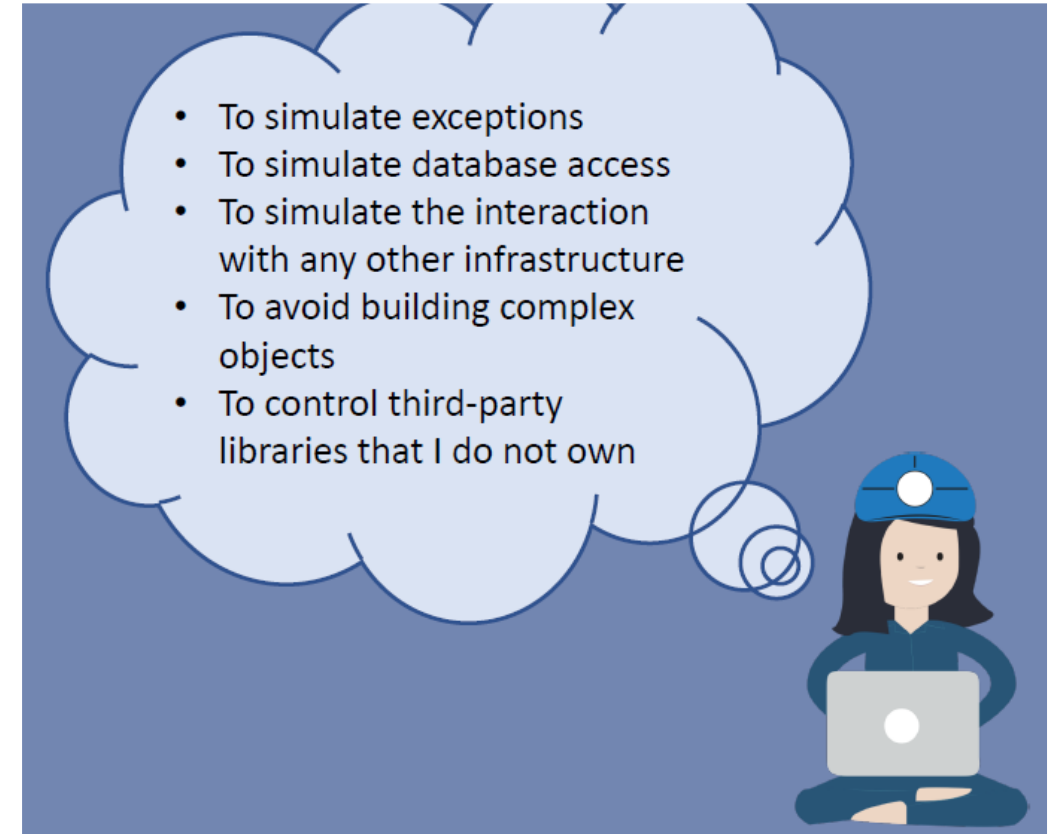
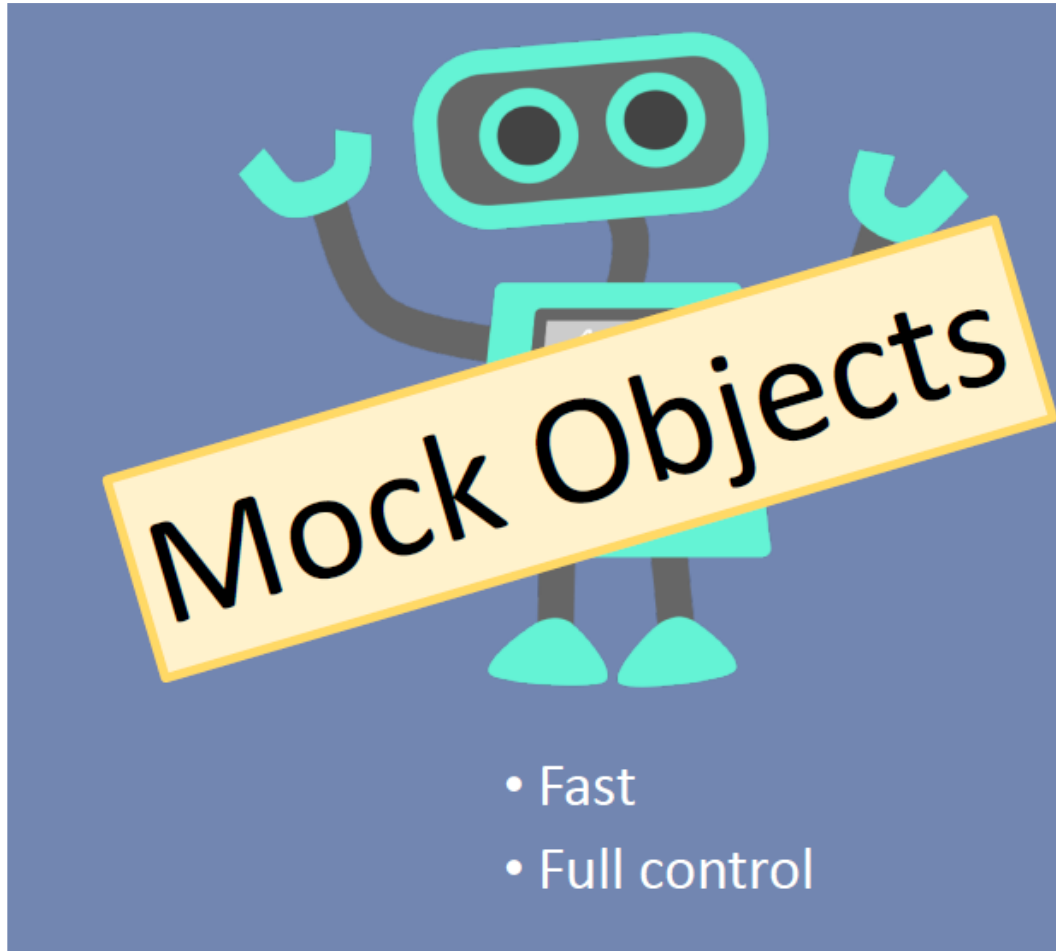
```
public List<Invoice> filter() {  
    InvoiceDao dao = new InvoiceDao();  
    List<Invoice> all= dao.all();  
    List<> filtered = new ArrayList<>();  
  
    for(Invoice inv : all) {  
        if(inv.getValue() < 100.0)  
            filtered.add(inv);  
    }  
  
    return filtered;  
}
```

This is what
we want to
test!

How can I get rid
of the database
for the sake of the
test?



Mocks and Mockito



Mocks and Mockito



➤ Mockito Official Site

➤ Java –Documentation of Mockito

Installation of Mockito

- Create a maven Project in the IDE.
- Add the latest dependency of “mockito-core” in POM file
[You can also find the dependency by going to [maven repository](#) -> typing mockito-core -> click the latest version -> copy the dependency]

```
<dependency>  
  <groupId>org.mockito</groupId>  
  <artifactId>mockito-core</artifactId>  
  <version>....</version>  
  <scope>test</scope>  
</dependency>
```

- Also add the following dependency of Junit5. [You can also get the dependency of Junit 5 from Maven Repository in the same way]

```
<dependency>  
  <groupId>org.junit.jupiter</groupId>  
  <artifactId>junit-jupiter-engine</artifactId>  
  <version>....</version>  
  <scope>test</scope>  
</dependency>
```